



Rapport de Projet Web



StudyByte

Plateforme de réservation d'espaces coworking

Mohamed Amine Tekaya
Rayen Raddaoui
Anis Othmen
Amen Allah Sahli

Année universitaire 2025–2026

Établissement : Institut Supérieur d'Informatique et de Mathématique de Monastir

Enseignantes :
Mme Senda LAÂTAWI
Mme Dorra DHAOU

7 avril 2026

Table des matières

1	Introduction et présentation de l'idée	3
1.1	Contexte et objectifs	3
1.2	L'idée originale	3
1.3	Défense de l'idée (justification)	4
2	Conception	5
2.1	Diagramme des cas d'utilisation (UC) principal	5
2.2	Détail des cas d'utilisation (UC détaillés)	6
2.3	Diagrammes de séquence	8
2.3.1	Séquence : Inscription d'un client	8
2.3.2	Séquence : Réservation d'une place	8
2.4	Diagrammes supplémentaires	9
2.4.1	Diagramme de classes	9
2.5	Architecture technique (front-end / back-end / BD)	9
2.6	Maquettes / wireframes	10
3	Développement	14
3.1	Technologies choisies et justifications	14
3.2	Organisation du code	14
3.3	Implémentation des fonctionnalités clés	15
3.3.1	Authentification et chiffrement du mot de passe	15
3.3.2	Chargement dynamique des places occupées	15
3.3.3	Insertion d'une réservation avec calcul de l'heure de fin	16
3.3.4	Tableau de bord administrateur avec filtres	16
3.4	Gestion des données	17
3.4.1	Modèle de données	17
3.4.2	Requête importante	18
4	Tests et validation	19
4.1	Environnement et outils de test	19
4.2	Scénarios de test	20
5	Conclusion et perspectives	21

5.1	Bilan du projet	21
5.2	Difficultés globales et apprentissages	21
5.3	Évolutions possibles	21
6	Annexes	23
6.1	Lien vers le dépôt Git	23
6.2	Captures d'écran de l'application	23
6.3	Fichiers de configuration	27

Chapitre 1

Introduction et présentation de l'idée

1.1 Contexte et objectifs

Dans le cadre de notre module de développement web, nous avons réalisé un projet de gestion et de réservation d'espaces de coworking intitulé **StudyByte**. Ce projet s'inscrit dans un contexte où la demande pour des espaces de travail flexibles et partagés est en pleine croissance, notamment auprès des étudiants, des freelances et des télétravailleurs.

L'objectif principal est de proposer une solution web complète permettant à des utilisateurs de consulter la disponibilité des places, de créer un compte, de se connecter et de réserver un poste de travail avec choix de la boisson, de l'horaire et de la durée de session. Le personnel administrateur dispose quant à lui d'un tableau de bord dédié pour le suivi en temps réel des réservations.

1.2 L'idée originale

StudyByte est une application web de réservation de postes de travail dans un espace co-working nommé *Espace VIP*. Elle résout le problème de la gestion manuelle et désorganisée des places dans ce type d'environnement.

Les fonctionnalités clés sont les suivantes :

- Visualisation interactive du plan de salle (zones Alpha, Beta, Gamma) avec indicateurs de disponibilité en temps réel.
- Inscription et authentification sécurisée des clients (chiffrement César +3 côté client).
- Réservation complète avec choix de la place, de la date, de l'heure, de la durée et de la boisson.
- Tableau de bord administrateur filtrable avec statistiques.
- Déconnexion propre pour tous les rôles.

1.3 Défense de l'idée (justification)

Ce projet a été choisi pour sa pertinence pratique : les espaces de coworking se multiplient en Tunisie et les gestionnaires manquent d'outils simples pour les administrer. Notre solution est légère (aucun framework front-end lourd), rapide à déployer grâce à **Supabase** comme back-end-as-a-service, et offre une expérience utilisateur intuitive avec un design soigné en HTML/CSS/JavaScript vanilla. Elle constitue également une excellente vitrine des compétences acquises en développement full-stack.

Chapitre 2

Conception

2.1 Diagramme des cas d'utilisation (UC) principal

Le diagramme suivant présente une vue d'ensemble des deux acteurs principaux — le **Client** et l'**Administrateur** — ainsi que leurs interactions avec le système StudyByte.

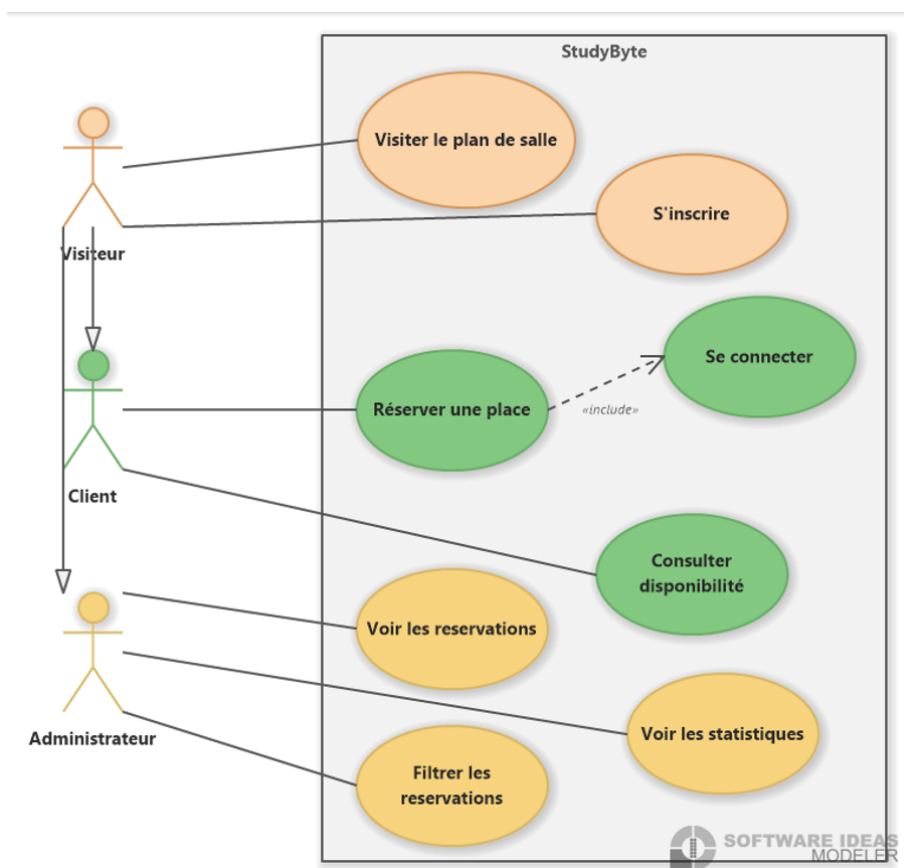


FIGURE 2.1 – Diagramme des cas d'utilisation général – StudyByte

Les acteurs sont :

- **Client (Visiteur / Utilisateur connecté)** : peut consulter les places, créer un compte, se connecter, effectuer une réservation et se déconnecter.
- **Administrateur** : peut se connecter avec des identifiants dédiés et consulter/filtrer toutes les réservations depuis le tableau de bord.

2.2 Détail des cas d'utilisation (UC détaillés)

Les trois cas d'utilisation les plus représentatifs de notre application sont décrits ci-dessous. La démarche d'enchaînement des interfaces est la suivante : l'utilisateur arrive sur la page d'accueil (`index.html`) pour visualiser le plan, clique sur « Réserver une place », est redirigé vers la page de connexion (`login.html`), crée éventuellement un compte (`signup.html`), puis accède au formulaire de réservation (`place.html`).

UC1 : S'inscrire

Préconditions	L'utilisateur n'a pas encore de compte. Il accède à la page <code>signup.html</code> .
Postconditions	Un enregistrement est créé dans la table <code>Client</code> (Supabase) avec le mot de passe chiffré. L'utilisateur est redirigé vers <code>login.html</code> .
Scénario nominal	1. L'utilisateur saisit son nom complet, son email, son mot de passe et la confirmation. 2. Le système valide les champs (longueur du nom, format email, longueur du mot de passe, correspondance des mots de passe). 3. Le système vérifie que l'email n'est pas déjà enregistré via Supabase. 4. Le mot de passe est chiffré (César +3) puis inséré dans la base de données. 5. L'utilisateur est redirigé vers la page de connexion.
Scénarios alternatifs	A1 : Un champ est vide → message d'erreur affiché. A2 : L'email existe déjà → message « Cet email est déjà utilisé ». A3 : Les mots de passe ne correspondent pas → message d'erreur.

UC2 : Se connecter

Préconditions	L'utilisateur possède un compte (client ou admin). Il est sur login.html.
Postconditions	Le rôle et l'email sont stockés dans localStorage. L'utilisateur est redirigé selon son rôle (admin.html ou index.html/place.html).
Scénario nominal	1. L'utilisateur sélectionne son type de compte (Client ou Administrateur). 2. Il saisit son email et son mot de passe. 3. Pour le rôle admin : comparaison avec les identifiants codés en dur (admin@espacevip.tn / Admin123!). 4. Pour le rôle client : interrogation de Supabase, comparaison du hash César +3. 5. En cas de succès : stockage en localStorage et redirection.
Scénarios alternatifs	A1 : Email non trouvé → « Adresse e-mail client non trouvée ». A2 : Mot de passe incorrect → « Mot de passe incorrect ». A3 : Champs vides → message d'erreur générique.

UC3 : Réserver une place

Préconditions	L'utilisateur est connecté (rôle user). Il est sur place.html. Les places occupées sont chargées depuis Supabase.
Postconditions	Une ligne est insérée dans la table reservations avec l'ensemble des informations. La place devient occupée dans l'interface.
Scénario nominal	1. L'utilisateur visualise le plan interactif et sélectionne une place libre. 2. Il renseigne son nom, email, téléphone, sa boisson préférée, la date, l'heure et la durée. 3. L'heure de fin est calculée automatiquement et affichée. 4. Il clique sur « Confirmer la réservation ». 5. Le système insère l'enregistrement dans Supabase et recharge les places occupées.
Scénarios alternatifs	A1 : Aucune place sélectionnée → alerte « Choisissez une place ». A2 : Champs obligatoires manquants → alerte de validation. A3 : Erreur Supabase → message d'erreur technique affiché.

2.3 Diagrammes de séquence

2.3.1 Séquence : Inscription d'un client

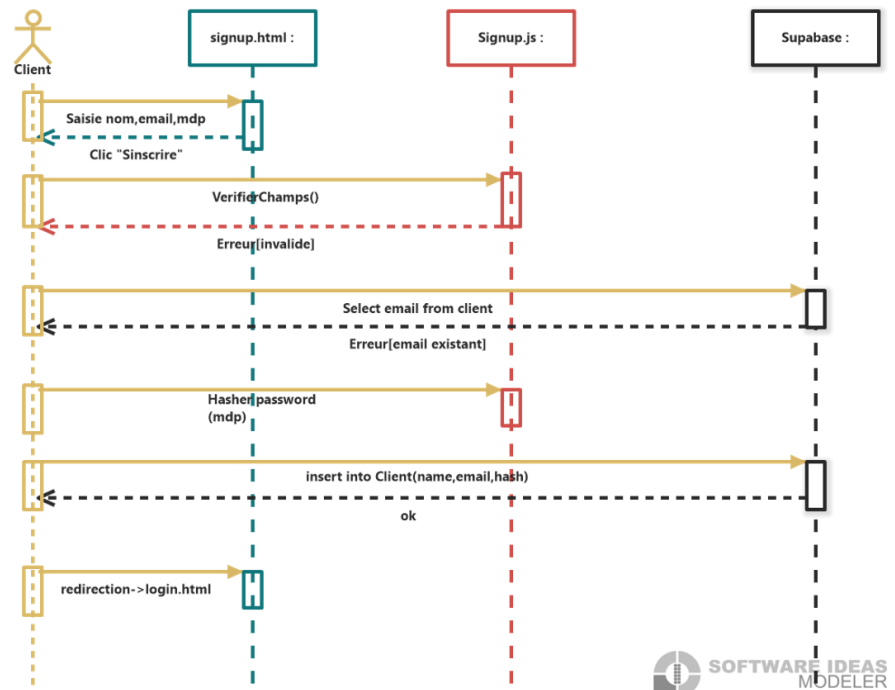


FIGURE 2.2 – Diagramme de séquence – Inscription d'un client

2.3.2 Séquence : Réservation d'une place

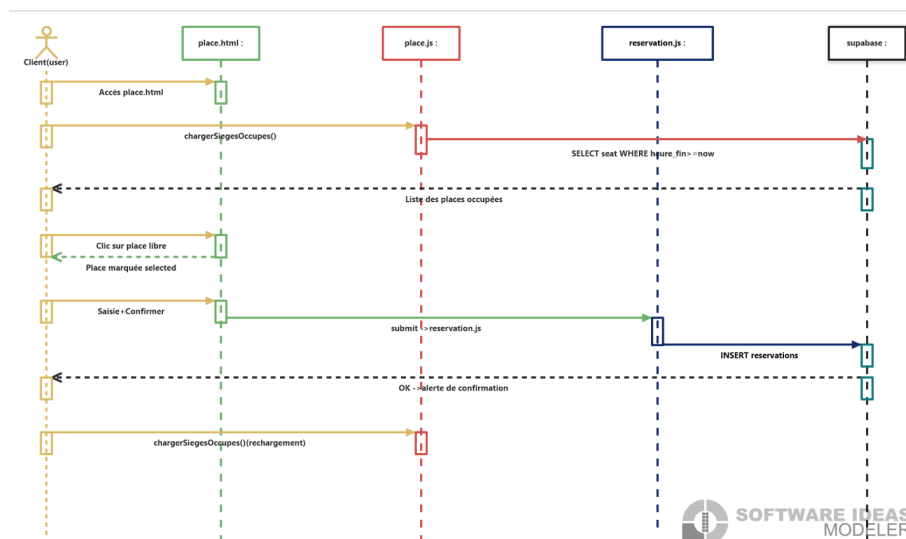


FIGURE 2.3 – Diagramme de séquence – Réservation d'une place

2.4 Diagrammes supplémentaires

2.4.1 Diagramme de classes

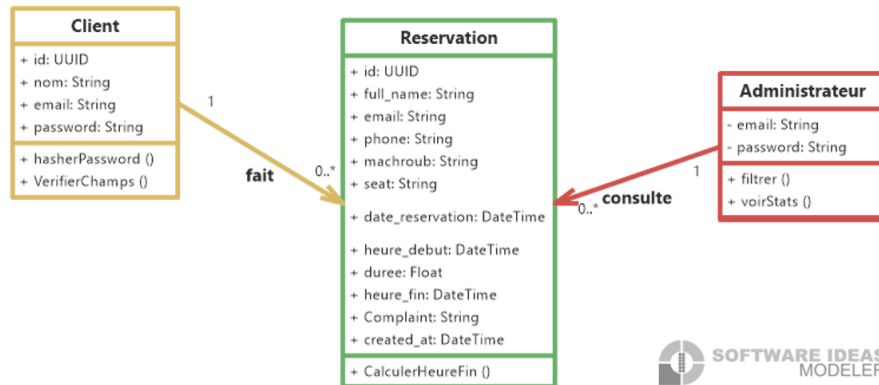


FIGURE 2.4 – Diagramme de classes – StudyByte

Les principales entités identifiées sont :

- **Client**: id, name, email, password (chiffré)
- **Reservation**: id, full_name, email, phone, machroub, seat, date_reservation, heure_debut, duree, heure_fin, complaint, created_at
- **Administrateur** : identifiants statiques (email + mot de passe)

2.5 Architecture technique (front-end / back-end / BD)

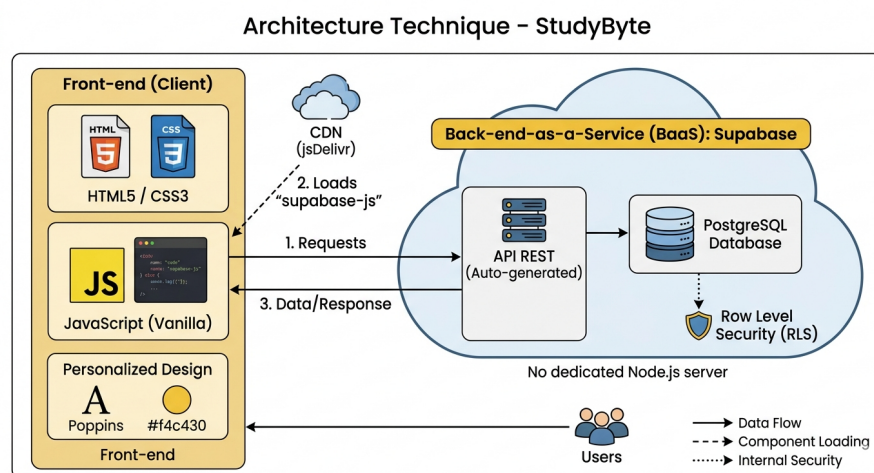


FIGURE 2.5 – Architecture technique – StudyByte

2.6 Maquettes / wireframes

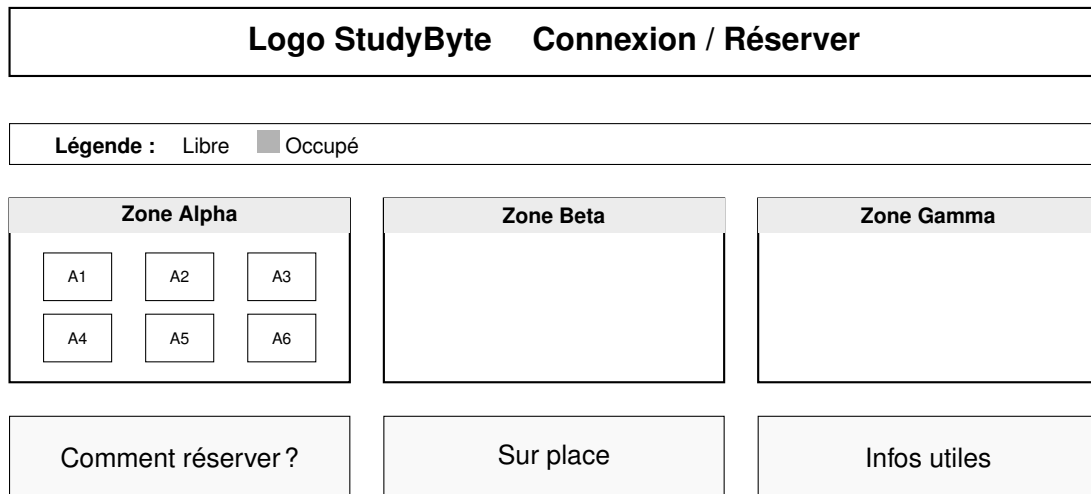


FIGURE 2.6 – Wireframe de la page d'accueil (index.html)

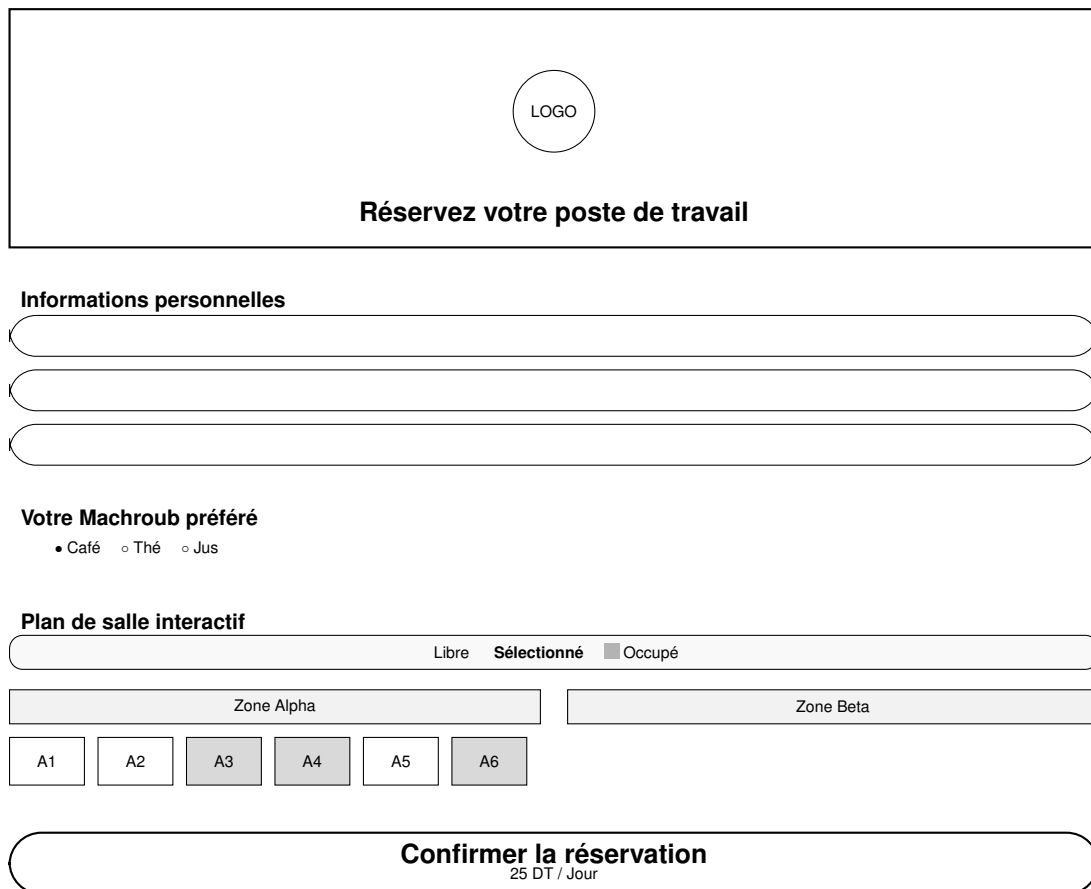


FIGURE 2.7 – Wireframe de la page de réservation (place.html)

LOGO

StudyByte

Bienvenue

Connectez-vous pour réserver une place ou accéder à l'administration.

Email

flenbenfoulen@gmail.com

Mot de passe

.....

SE CONNECTER

Pas de compte ? **S'inscrire**

FIGURE 2.8 – Wireframe de la page de connexion

LOGO

StudyByte

Créer un compte

Rejoignez-nous dès maintenant !

Nom complet

Flen Ben Foulén

Email

flenbenfoulén@gmail.com

Mot de passe

.....

Confirmer le mot de passe

.....

S'INSCRIRE

Vous avez déjà un compte ? **Se connecter**

FIGURE 2.9 – Wireframe de la page d’inscription (signup.html)

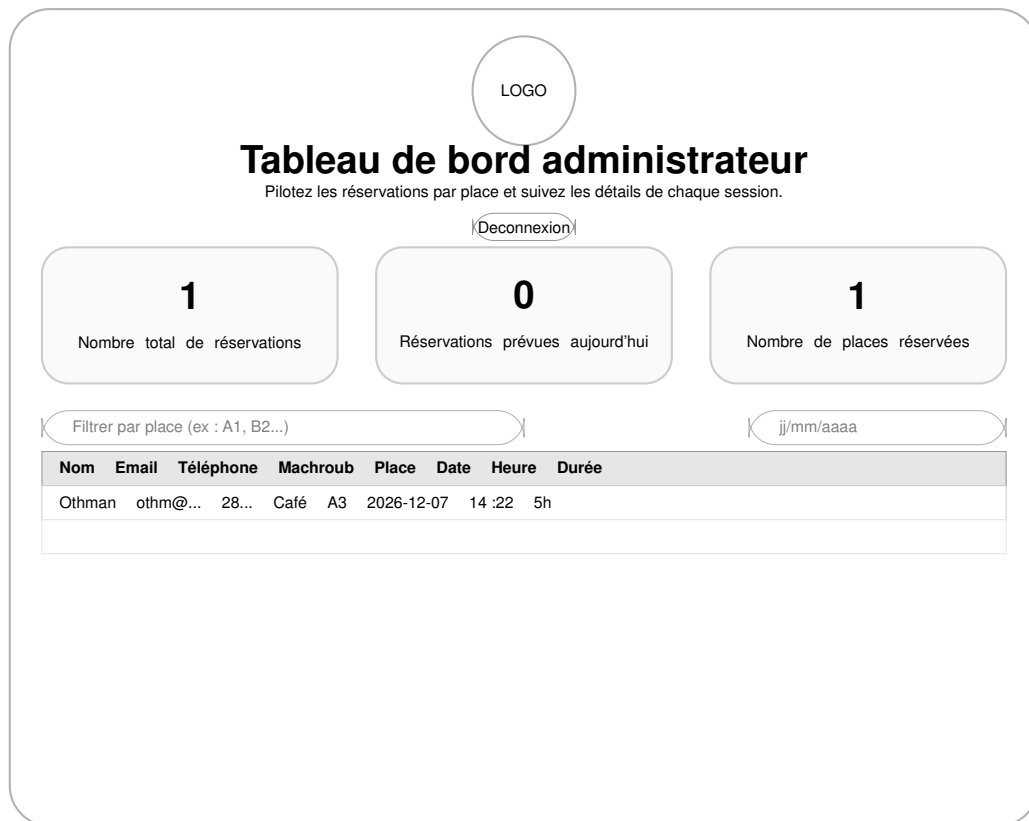


FIGURE 2.10 – Wireframe du tableau de bord administrateur (admin.html)

Chapitre 3

Développement

3.1 Technologies choisies et justifications

- **Front-end : HTML5 / CSS3 / JavaScript (vanilla)** – Choix délibéré de ne pas utiliser de framework pour garder le projet léger, maîtriser chaque ligne de code et ne pas alourdir les dépendances. Le design est entièrement personnalisé (police Poppins, palette dorée #f4c430).
- **Back-end-as-a-Service : Supabase (PostgreSQL)** – Supabase fournit une base de données PostgreSQL hébergée, une API REST auto-générée et une bibliothèque JavaScript. Cela élimine le besoin d'un serveur Node.js dédié, simplifie le déploiement et accélère le développement.
- **CDN** – Les dépendances externes (supabase-js) sont chargées via jsdelivr.net, sans étape de build, ce qui facilite le démarrage du projet.

3.2 Organisation du code

L'arborescence du projet est la suivante :

```
1 projet-web/  
2   back/  
3     admin.js           # Logique du tableau de bord admin  
4     login.js           # Authentification (client + admin)  
5     place.js           # Chargement des sieges occupés  
6     reservation.js     # Soumission du formulaire de réservation  
7     signup.js          # Inscription et hashage du mot de passe  
8     supabase-client.js # Initialisation du client Supabase  
9   front/  
10    admin.css           # Styles du dashboard administrateur  
11    admin.html          # Page du tableau de bord administrateur  
12    index.html          # Page d'accueil (plan général)  
13    login.css           # Styles de la page de connexion  
14    login.html          # Page de connexion  
15    logo.png            # Logo Espace VIP  
16    place.css           # Styles de la page de réservation
```



```

17   place.html           # Formulaire de reservation
18   signup.css           # Styles de la page d'inscription
19   signup.html          # Page d'inscription
20   README.md

```

Listing 3.1 – Arborescence du projet StudyByte

La séparation back/ / front/ reflète le principe de séparation des préoccupations : la logique métier (appels Supabase, validation, routage) est isolée dans des fichiers JavaScript dédiés, tandis que la présentation est gérée par les fichiers HTML/CSS.

3.3 Implémentation des fonctionnalités clés

3.3.1 Authentification et chiffrement du mot de passe

Logique : Pour éviter de stocker les mots de passe en clair dans Supabase, un chiffrement de substitution de type César +3 est appliqué avant l’insertion ou la comparaison.

Difficulté rencontrée : Supabase ne fournit pas nativement de mécanisme d’authentification par email/password custom avec hachage personnalisé. La solution adoptée est de gérer soi-même la table Client et de comparer les mots de passe côté client après chiffrement.

```

1 function hasherPassword(plainPassword) {
2   let result = "";
3   for (let i = 0; i < plainPassword.length; i++) {
4     result += String.fromCharCode(plainPassword.charCodeAt(i) + 3);
5   }
6   return result;
7 }

```

Listing 3.2 – Fonction de chiffrement César +3 (signup.js / login.js)

3.3.2 Chargement dynamique des places occupées

Logique : A l’ouverture de place.html, une requête Supabase récupère toutes les réservations dont heure_fin est supérieure à l’instant présent, ce qui correspond aux réservations encore actives.

```

1 async function chargerSiegesOccupes() {
2   const now = new Date().toISOString();
3   const { data, error } = await db
4     .from('reservations')
5     .select('seat')
6     .gte('heure_fin', now);
7
8   if (error) { console.error(error); return; }
9 }

```

```

10     document.querySelectorAll('.seat').forEach(s => {
11         s.classList.remove('occupied', 'selected');
12         s.classList.add('free');
13     });
14
15     data.forEach(r => {
16         const seat = document.querySelector('[data-seat="${r.seat}"]');
17         if (seat) {
18             seat.classList.remove('free');
19             seat.classList.add('occupied');
20         }
21     });
22     attacherClics();
23 }

```

Listing 3.3 – Chargement des sieges occupes (place.js)

3.3.3 Insertion d'une réservation avec calcul de l'heure de fin

Logique : L'heure de fin est calculée côté client en ajoutant la durée (en heures, décimales acceptées) à l'heure de début, puis convertie en ISO 8601 avant insertion.

```

1 const heureFin = new Date(`${date}T${heure}:00Z`);
2 heureFin.setHours(heureFin.getHours() + Math.floor(duree));
3 heureFin.setMinutes(heureFin.getMinutes() + (duree % 1) * 60);
4
5 const { error } = await db.from('reservations').insert({
6     full_name      : nom,
7     email          : email,
8     phone          : telephone,
9     machroub       : boisson,
10    seat           : seat,
11    date_reservation: date,
12    heure_debut     : heure,
13    duree           : duree,
14    complaint       : reclamation || null,
15    heure_fin       : heureFin.toISOString()
16 });

```

Listing 3.4 – Calcul heure de fin et insertion Supabase (reservation.js)

3.3.4 Tableau de bord administrateur avec filtres

Logique : L'admin charge toutes les réservations triées par date décroissante, puis filtre côté client selon le numéro de place et la date saisie. Les statistiques (total, aujourd'hui, places distinctes) se recalculent à chaque filtre.

```

1 function applyFilters() {
2     const placeQuery = (searchInput.value || '').trim().toUpperCase();
3     const selectedDate = dateFilter.value;
4
5     const filtered = reservations.filter((item) => {
6         const seat = (item.seat || '').toUpperCase();
7         const byPlace = !placeQuery || seat.includes(placeQuery);
8         const byDate = !selectedDate || item.date_reservation ===
9             selectedDate;
10        return byPlace && byDate;
11    });
12    renderTable(filtered);
13    renderStats(filtered);
14 }

```

Listing 3.5 – Filtrage des réservations (admin.js)

3.4 Gestion des données

3.4.1 Modèle de données

Deux tables PostgreSQL sont utilisées dans Supabase :

TABLE 3.1 – Table Client

Colonne	Type	Description
id	UUID (PK)	Identifiant auto-généré
name	TEXT	Nom complet du client
email	TEXT UNIQUE	Adresse email (identifiant de connexion)
password	TEXT	Mot de passe chiffré (César +3)

TABLE 3.2 – Table reservations

Colonne	Type	Description
id	UUID (PK)	Identifiant auto-généré
full_name	TEXT	Nom du réservant
email	TEXT	Email du réservant
phone	TEXT	Téléphone (optionnel)
machroub	TEXT	Boisson choisie
seat	TEXT	Numéro de la place (ex : A1, B3)
date_reservation	DATE	Date de la réservation
heure_debut	TIME	Heure de début
duree	NUMERIC	Durée en heures
heure_fin	TIMESTAMPTZ	Heure de fin (calculée, ISO 8601)
complaint	TEXT	Réclamation (optionnel)
created_at	TIMESTAMPTZ	Date de création (auto)

3.4.2 Requête importante

Récupération des réservations encore actives pour déterminer les places occupées :

```

1 SELECT seat
2 FROM reservations
3 WHERE heure_fin >= NOW();

```

Listing 3.6 – Requete Supabase – Places occupees

Récupération de toutes les réservations pour l’administrateur, triées par date et heure décroissantes :

```

1 SELECT id, full_name, email, phone, machroub, seat,
2     date_reservation, heure_debut, duree, heure_fin,
3     complaint, created_at
4 FROM reservations
5 ORDER BY date_reservation DESC, heure_debut DESC;

```

Listing 3.7 – Requete Supabase – Dashboard admin

Chapitre 4

Tests et validation

4.1 Environnement et outils de test

Les tests ont été effectués sur les navigateurs suivants :

- Google Chrome (version récente) – navigateur principal de développement
- Mozilla Firefox – tests de compatibilité CSS
- Microsoft Edge – vérification du comportement du `localStorage`

Les outils utilisés incluent les DevTools de Chrome (onglets Console, Network, Application) pour inspecter les appels Supabase, les erreurs JavaScript et le contenu du `localStorage`.

4.2 Scénarios de test

TABLE 4.1 – Résultats des tests fonctionnels – StudyByte

Fonctionnalité	Statut	Observations
Affichage du plan (index.html)	OK	Places toutes libres en mode visiteur
Inscription client	OK	Validation des champs + hash César
Connexion client	OK	Redirection selon rôle
Connexion administrateur	OK	Identifiants hardcodés fonctionnels
Chargement des places occupées	OK	Requête <code>heure_fin >= NOW()</code> correcte
Sélection d'une place	OK	Une seule place sélectionnable à la fois
Réservation complète	OK	Insertion Supabase + rechargement UI
Calcul de l'heure de fin	OK	Durées décimales gérées (ex : 1.5 h)
Dashboard admin – affichage	OK	Toutes les colonnes visibles
Dashboard admin – filtre par place	OK	Filtre insensible à la casse
Dashboard admin – filtre par date	OK	Résultats filtrés correctement
Déconnexion (tous rôles)	OK	localStorage vidé, redirection login
Redirection si non connecté	OK	Accès à <code>place.html/admin.html</code> refusé
Responsive mobile	Partiel	Adaptations CSS présentes, testé sur Chrome DevTools

Chapitre 5

Conclusion et perspectives

5.1 Bilan du projet

Le projet StudyByte a atteint ses objectifs principaux : une application web fonctionnelle de réservation d’espaces coworking, couvrant l’ensemble du cycle utilisateur – de l’inscription à la réservation en passant par l’authentification – ainsi qu’un tableau de bord administrateur opérationnel.

La base de données Supabase s’est révélée particulièrement efficace pour ce type de projet : elle élimine le besoin d’un serveur back-end dédié, fournit une API REST robuste et permet une mise en production rapide. Le choix de JavaScript vanilla a permis de maîtriser chaque aspect du code sans dépendance superflue.

5.2 Difficultés globales et apprentissages

Les principales difficultés rencontrées ont été :

- La gestion de la synchronisation entre le `localStorage` et l’état réel de Supabase (places occupées en temps réel).
- La sécurité côté client : un chiffrement César reste très faible en production ; cette contrainte était connue et acceptée dans le cadre pédagogique du projet.
- La gestion des fuseaux horaires dans les comparaisons ISO 8601 avec `heure_fin >= NOW()`.

Compétences acquises : intégration d’un BaaS (Supabase), manipulation avancée du DOM, gestion des rôles utilisateurs, design responsive, organisation d’un projet web multi-fichiers.

5.3 Évolutions possibles

- Remplacement du chiffrement César par un vrai hachage (bcrypt côté serveur ou utilisation de l’authentification Supabase Auth).
- Ajout d’un système de notifications par email (confirmation de réservation).
- Mise en place d’un système de paiement en ligne (intégration Stripe).

- Application mobile (React Native ou Flutter) utilisant la même API Supabase.
- Déploiement sur Vercel ou Netlify avec domaine personnalisé.

Chapitre 6

Annexes

6.1 Lien vers le dépôt Git

<https://github.com/anisothman/projet-web>

6.2 Captures d'écran de l'application

Page d'accueil – index.html

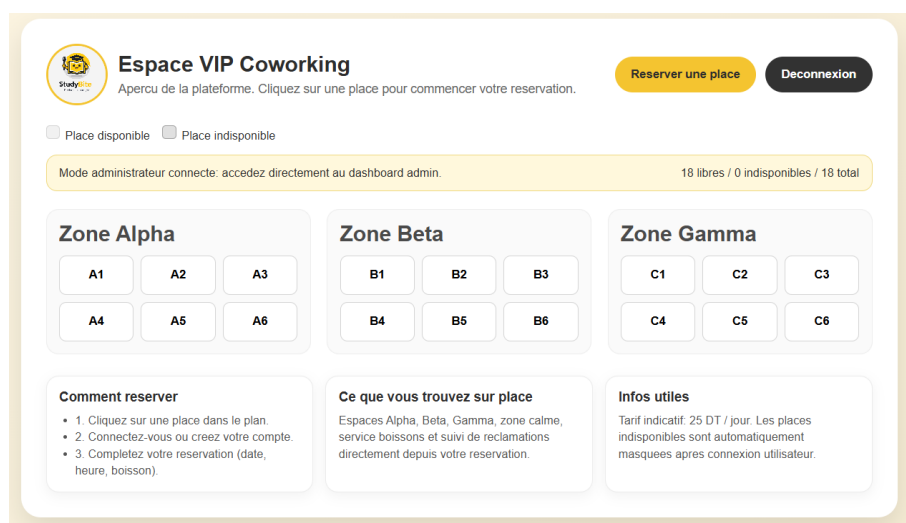


FIGURE 6.1 – Interface d'accueil – Plan des places

Page de connexion – login.html

Bienvenue

Connectez-vous pour réserver une place ou accéder à l'administration.

Email

Mot de passe

SE CONNECTER

Pas de compte ? [S'inscrire](#)

FIGURE 6.2 – Interface de connexion

Page d'inscription – signup.html

Créer un compte

Rejoignez-nous dès maintenant!

Nom complet

Flen Ben Foulen

Email

flenbenfoulen@gmail.com

Mot de passe

Confirmer le mot de passe

S'INSCRIRE

Vous avez déjà un compte ? [Se connecter](#)

FIGURE 6.3 – Interface d'inscription

Page de réservation – place.html

StudyByte

Deconnexion

ESPACE VIP

Réservez votre poste de travail

Un environnement spacieux conçu pour votre productivité.

Informations personnelles

Nom complet

Adresse Email

Téléphone

Votre Machroub préféré

☒ Café ☐ Thé ☐ Jus

Réclamation ou remarque (optionnel)

Partagez votre remarque ici...

Date, heure et durée de réservation

jj/mm/aaaa

...

la durée en h

Plan de salle interactif

☐ Disponible ☒ Sélectionné ☐ Occupé

Zone Alpha

A1 A2 A3 A4 A5 A6 A7

Zone Beta

B1 B2 B3 B4 B5 B6

Zone Gamma

C1 C2 C3 C4 C5 C6

Confirmer la réservation

25 DT / Jour

FIGURE 6.4 – Formulaire de réservation avec plan interactif

Tableau de bord administrateur – admin.html



FIGURE 6.5 – Tableau de bord administrateur

6.3 Fichiers de configuration

```

1 const supabaseUrl = 'https://grelnipffaizijgpzhli.supabase.co';
2 const supabaseKey = 'eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...';
3 const db = supabase.createClient(supabaseUrl, supabaseKey);

```

Listing 6.1 – supabase-client.js – Initialisation de la connexion Supabase

Note : la clé API publique (anon key) est destinée à un usage côté client uniquement. Les règles de sécurité (Row Level Security) de Supabase doivent être configurées pour restreindre les accès en production.